

DAY-15 — SQL Fundamentals

CyberNotes • Detailed Beginner Reference (Improved Edition)

Learning goal: Build a practical, working understanding of **sql fundamentals**, connect the concept to real cybersecurity work, and practice it safely in a local lab.

Study rule: Understand a command or concept before copying it. Security tools are powerful because they interact with real systems — your practice environment should always be intentionally controlled and authorized.

1. Core Concepts

Core idea: SQL Fundamentals is a building block for practical cybersecurity. This section explains what each term actually means, where you'll encounter it, and how it connects to the bigger picture.

Why it matters: Security work depends on understanding normal system and network behavior before you can recognize abnormal behavior. This topic connects directly to Linux, networking, web security, monitoring, and troubleshooting — the threads running through this whole 20-day series.

Vocabulary covered: databases, tables, rows, keys, SELECT, WHERE, JOIN, INSERT.

Reference	What to remember
SELECT	read rows
WHERE	filter rows
ORDER BY	sort
JOIN	combine tables
INSERT	add rows
UPDATE	change rows
DELETE	remove rows
CREATE TABLE	define schema

2. Concept Explanations

Each term below is explained in plain language, with a note on why it matters for security work. Read these before running any commands.

1. databases

A database is an organized collection of structured data, managed by a database engine (MySQL, PostgreSQL, SQLite, etc.) that handles storage, retrieval, and concurrent access. Nearly every application with user accounts, transactions, or persistent data relies on one, which is exactly why database security matters so much.

2. tables

A table stores data in rows and columns, similar to a spreadsheet, with each column defined to hold a specific type of data (text, number, date). Well-designed tables reflect the real-world things they represent — a `users` table, an `orders` table — connected to each other through keys.

3. rows

A row (record) is a single entry in a table — one user, one order, one log event. Reading, adding, changing, or deleting specific rows is the fundamental unit of work in almost every database operation.

4. keys

A primary key uniquely identifies each row in a table (like a user ID), while a foreign key references a primary key in another table to link related data together — an order's foreign key might point to the user who placed it. Keys are what make relational databases "relational," and what makes JOIN possible.

5. SELECT

`SELECT` retrieves data from one or more tables — the most frequently used SQL statement, forming the basis of nearly every query. `SELECT \* FROM users` reads every column of every row in the users table.

6. WHERE

`WHERE` filters which rows a query returns based on a condition, e.g. `SELECT \* FROM users WHERE username = 'alice'`. This is exactly the clause that SQL injection attacks (Day 16) target — by manipulating what gets inserted into the WHERE condition, an attacker can change what the query actually filters for.

7. JOIN

`JOIN` combines rows from two or more tables based on a related key, e.g. joining `orders` to `users` to see which user placed each order. Understanding joins is essential for reading any non-trivial real-world query.

8. INSERT

`INSERT` adds new rows to a table, e.g. `INSERT INTO users (name, email) VALUES ('Bob', 'bob@example.com')`. Applications typically build these statements dynamically from user-submitted form data — which is precisely where unsafe query construction becomes a security risk.

3. Safe Practical Lab

Step 1. Define the lab target and the exact question you are trying to answer.

Step 2. Run the smallest safe observation or command that can answer that question.

Step 3. Read the output carefully; record important fields, errors, timestamps, and assumptions.

Step 4. Change one variable at a time and compare the result.

Step 5. Write a short conclusion and clean up temporary files or services.

Example lab sequence:

```
Create a small local database
SELECT rows
filter with WHERE
JOIN two related tables
inspect results
```

4. Common Mistakes & Troubleshooting

- Running a command without checking its target, current directory, or privileges.
- Confusing a tool's output with proof of a vulnerability or compromise — output is evidence, not a conclusion.
- Ignoring error messages instead of reading the exact reason for failure.
- Changing several settings at once, making it hard to tell what actually fixed or broke the lab.
- Saving real credentials, tokens, or personal data in practice notes or public repositories.
- Testing a public target just because it's reachable — reachability is never the same as authorization.

Troubleshooting checklist:

- What system am I on?
- What exact command/request did I run?
- What did I expect?
- What happened instead?
- What does the error message say?
- Can I reproduce it in a clean local lab?
- What changed since the last successful attempt?

5. Deep-Dive Questions

Answer these in your own words after finishing the lab — they're designed to check real understanding, not memorization.

Q1. What problem does databases solve, in your own words?

Notes: _____

Q2. What is the difference between normal behavior and a suspicious observation in this topic?

Notes: _____

Q3. Which command, log, packet, file, or response would you inspect first, and why?

Notes: _____

Q4. What could cause a misleading result or false positive here?

Notes: _____

Q5. How would you document your observation so another learner could reproduce it safely?

Notes: _____

Q6. Which security control reduces the risk associated with this topic?

Notes: _____

6. Revision Checklist

- I can explain sql fundamentals in my own words.
- I can identify the main terms without looking at the notes.
- I can perform the example in a controlled lab.
- I can interpret the output instead of only copying it.
- I know at least one defensive or troubleshooting use for the concept.
- I can document the result clearly and safely.

7. Safety & Scope

Use these commands and tools only on systems you own, a local virtual machine, a CTF, an intentionally vulnerable application, or a target for which you have explicit written permission.

Keep a written scope: target, allowed actions, time window, and stop conditions. Never scan or test random public systems.

When practicing, save your notes and outputs so you can explain what you observed rather than blindly running commands.

Next step: After Day-15, review your notes and connect this topic to the next subject in the series. The goal is not to memorize commands — it's to understand systems well enough to reason about security.

8. Quick Reference & Personal Notes

Topic: Day-15 — SQL Fundamentals

Three things I learned:

1. _____
1. _____
1. _____

Commands / concepts to memorize:

- SELECT — read rows
- WHERE — filter rows
- ORDER BY — sort
- JOIN — combine tables
- INSERT — add rows
- UPDATE — change rows
- DELETE — remove rows
- CREATE TABLE — define schema

My lab result

What I tested: _____

What I observed: _____

What I would investigate next: _____

Final reminder: A strong cybersecurity learner can explain both the offensive concept and the defensive reason for learning it. Keep practice authorized, reproducible, and documented.